

Matching Engine / Order Book Classes

Module Goal: Maintain Per-Symbol Order books, match aggressive orders against passive side, produce trades.

PriceLevel

Manages all orders at one specific price. On one side (bid or ask).

Order Queue

Pure FIFO Container (Ensures time priority)

Could be merged if OrderQueue is simple?

OrderBook

Manages one instrument's full bid & ask Sides.

(One order book per Symbol)
(Coarse-grained book (per book) for simplicity)

Matching Engine

Orchestrates matching across all order books.

* Core matching logic: Aggressive order walks passive side.
* Partial fills Supported.
* Thread-Safety: lock-per Symbol book during processing

PriceLevel

- double Price
- Side side
- `std::deque<std::shared_ptr<Order>>` orders // FIFO Queue
- double totalQty // cached sum
- `std::mutex` levelMutex

- + `void addOrder(std::shared_ptr<Order> order)`
- + `bool removeOrder(uint64_t orderId)`
- + `double getTotalQty()` const
- + `std::shared_ptr<Order> getFrontOrder()`

Order Queue

- `std::deque<std::shared_ptr<Order>>` queue
- `std::mutex` queueMutex

- + `void push(std::shared_ptr<Order> order)`
- + `std::shared_ptr<Order> pop()`
- + `std::shared_ptr<Order> front()` const
- + `size_t size()` const

Ensures time Priority (oldest order first)

Composes PriceLevel (owns them)

PriceLevel (has many orders)

Aggregates "Order" (via "activeOrder" map)

Composition with Order

Depends on TradePublisher for publishing matches

OrderBook

- `std::string` Symbol
- `std::map<double, std::shared_ptr<PriceLevel>, std::greater<double>>` bids // descending Prices
- `std::map<double, std::shared_ptr<PriceLevel>>` asks // ascending prices
- `std::unordered_map<uint64_t, std::shared_ptr<Order>>` activeOrders // 10 -> order lookup
- `std::mutex` bookMutex

- + `void addOrder(std::shared_ptr<Order> order)`
- + `bool cancelOrder(uint64_t orderId)`
- + `std::pair<double, double> getBestBidAsk()` const
- + `double getTotalBidQty()` const

Manages multiple OrderBook instances

MatchingEngine

- `std::unordered_map<std::string, std::shared_ptr<OrderBook>>` orderBooks
- `std::shared_ptr<TradePublisher>` tradePublisher

- + `void processNewOrder(std::shared_ptr<Order> order)`
- + `void processCancel(uint64_t orderId, const std::string & symbol)`
- + `void processReplace(uint64_t orderId, double newPrice, double newQty)`
- `std::vector<Trade> matchAggressive(std::shared_ptr<Order> aggressiveOrder)`